Software Requirements Specification (SRS)

Requirements Specification (SRS) of 25-26J-041

Project ID: **25-26J-041**

Project Title:

The Guardian: A Modular AI for Proactive Human-Layer Cybersecurity on Social Media

Student Details:

Names: Herath, H. M. G. D. K.
De Alwis, H. P. G. H. S.
Fernando, M. R. I.
Vishwa, J. R.

Student IDs: IT22361486
IT22317926
IT22366054
IT22916112

Supervisor: Mr. Kavinga Yapa Abeywardena

Date of Submission: [9/1/2026]

Contents

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a comprehensive and detailed description of the requirements for the development of "The Guardian" (Project ID: 25-26J-041). This document formally defines the functional and non-functional requirements, external interfaces, and design constraints necessary to guide the implementation and verification of the system.

This document is intended to serve as the primary reference for the project development team (consisting of four specialized module leads), the project supervisors, and potential stakeholders. It establishes a unified understanding of the system's scope, which includes a client-side browser extension and a centralized Python-FastAPI backend orchestrating four distinct AI-powered detection modules:

PII & OpSec Risk Detection Module

Misinformation Detection Module.

Hate Speech Detection Module.

Scam & Phishing Detection Module.

By documenting these requirements, this SRS aims to ensure that the final software solution meets the specified objectives of proactive, human-layer cybersecurity, privacy preservation, and non-intrusive user education.

1.2 Scope

This document covers the functional and non-functional requirements for **"The Guardian"**, a modular AI-powered cybersecurity system designed to protect social media users from human-layer threats.
The scope of this document encompasses the design, implementation, and integration of the following system components and functionalities:

- System Architecture:
    - **Frontend**: A lightweight, JavaScript-based browser extension compatible with major web browsers (e.g., Chrome, Firefox) that scrapes visible text from social media platforms in real-time.
    - **Backend**: A centralized API Gateway built with **FastAPI (Python)** that orchestrates data flow between the client and four specialized analysis modules.

- Core Detection Modules:
    1. **PII & OpSec Module**: Detection of unintentional self-disclosure of Personally Identifiable Information (PII) via **Named Entity Recognition (NER)** and **Regex**, as well as contextual Operational Security (OpSec) risks (e.g., announcing future travel plans).
    2. **Misinformation Module**: Identification of manipulative linguistic patterns (e.g., loaded language, emotional appeals) rather than simple fact-checking. This includes a **User Profiling** mechanism to tailor educational explanations based on the user's media literacy and background.
    3. **Hate Speech Module**: Detection of explicit hate speech and implicit, context-dependent toxicity (e.g., "dog whistles") using fine-tuned Transformers, ensuring explainability to the user.
    4. **Scam & Phishing Module**: A hybrid detection framework combining **Deterministic URL Heuristics** (path depth, subdomain abuse, and extension analysis) with **Context Aware AI Text Analysis**. The module utilizes **URL Tokenization** to force the AI to learn semantic intent and employs **Max Risk Logic** to ensure high-risk signals are not diluted. It also features **Incognito Isolation** for safe browsing.

- **User Interaction**: The document defines the requirements for a non-intrusive User Interface (UI) that uses subtle color-coded icons and interactive tooltips to "nudge" users toward safer behavior without censoring content or disrupting the browsing experience.
- **Data Privacy**: The document specifies privacy constraints, ensuring that the system operates stateless, where user text is analyzed in real-time without being permanently stored or logged by the backend.

1.3 Definitions, Acronyms, and Abbreviations

Comment: Glossary of that will be used throughout the documents, as well as in other documents based on this one. Usually in the form of table.

Acronym	Definition/Expansion	Context
API	Application Programming Interface	System Architecture
BERT	Bidirectional Encoder Representations from Transformers	AI / Core
Dog Whistle	Coded language used to communicate a specific message to a target group without provoking the general public.	Hate Speech Module
DOM	Document Object Model	Frontend
FastAPI	Python Web Framework	Backend
Homoglyph	A character that looks visually identical to another (used in URL spoofing).	Scam Module
HTTPS	Hypertext Transfer Protocol Secure	Communication
JSON	JavaScript Object Notation	Communication
NER	Named Entity Recognition	PII Module
NLP	Natural Language Processing	System Core

Nudge	A non-intrusive alert designed to influence user behavior toward safety.	UI / UX
OpSec	Operational Security	PII Module
Phishing	Fraudulent attempts to obtain sensitive information via social engineering.	Scam Module
PII	Personally Identifiable Information	PII Module
Safe Mode	An isolated environment for viewing suspicious URLs.	Scam Module
SRS	Software Requirements Specification	Documentation
TLS	Transport Layer Security	Security
Toxic Spans	Specific segments of text identified as hateful or offensive.	Hate Speech Module
UI	User Interface	Frontend
URL	Uniform Resource Locator	Scam Module

1.5 Overview

Software Overview "The Guardian" is a modular, AI-powered cybersecurity system designed to provide proactive, real-time protection against human-layer threats on social media platforms. The system functions as a client-server application, consisting of a lightweight browser extension frontend that scrapes visible text and a centralized Python-FastAPI backend that orchestrates analysis across four specialized detection modules.

Main Goals

- **Proactive Defense:** To identify potential threats—such as privacy leaks, manipulative content, or scams—before the user interacts with them or suffers harm.
- **Non-Intrusive Guidance:** To function as a cognitive "nudge" by providing subtle, color-coded alerts and educational tooltips rather than censoring or blocking content.
- **User Education:** To enhance the user's media literacy and personal security posture by explaining *why* specific content was flagged (e.g., identifying "dog whistles" or manipulative rhetoric).
- **Privacy Preservation:** To ensure user privacy by analyzing text in a stateless manner without storing personal data or requiring persistent login credentials.

Target Users The primary users of "The Guardian" are everyday social media users who may lack the technical expertise or time to critically evaluate every piece of content they encounter. This includes individuals vulnerable to financial fraud, online harassment, or misinformation campaigns, as well as those concerned about their digital privacy.

Key Tasks The system performs four primary detection tasks, managed by specialized modules in the following order:

1. **PII & OpSec Risk Detection:**
    - Detects unintentional self-disclosure of structured Personally Identifiable Information (PII) such as phone numbers and emails using Regex.
    - Identifies unstructured PII (names, locations) using Named Entity Recognition (NER).
    - Analyzes text for contextual Operational Security (OpSec) risks, such as combining location data with future temporal keywords to identify travel exposure.
2. **Misinformation Detection:**
    - Identifies manipulative linguistic patterns (e.g., loaded language, emotional appeals) rather than relying solely on fact-checking databases.
    - Utilizes a User Profiling mechanism to capture metadata (e.g., media literacy level) and tailor the complexity of the educational explanations provided to the user.
3. **Hate Speech Detection:**
    - Detects both explicit hate speech and implicit, context-dependent toxicity, including "dog whistles" and slang.
    - Provides explainable alerts that distinguish between offensive language and hate speech to reduce false positives on legitimate critiques.
4. **Scam & Phishing Detection:**
    - A hybrid detection framework combining **Deterministic URL Heuristics** (path depth, subdomain abuse, and extension analysis) with **Context Aware AI Text Analysis**.
    - The module utilizes **URL Tokenization** to force the AI to learn semantic intent and employs **Max Risk Logic** to ensure high-risk signals are not diluted. It also features **Incognito Isolation** for safe browsing.

2. Overall Descriptions

"The Guardian" is a standalone, client-server software system designed to function as a comprehensive security layer between social media platforms and the end-user. Unlike traditional cybersecurity tools that protect infrastructure or devices from malware, this software focuses on **Human-Layer Security**, mitigating threats that exploit psychological vulnerabilities, cognitive biases, and lack of awareness.

The system operates independently of the social media platforms it monitors, ensuring that it can provide unbiased, privacy-preserving analysis without relying on the platforms' internal moderation policies. It is composed of two primary architectural components:

- **The Client (Browser Extension):** A lightweight frontend compatible with major web browsers (e.g., Chrome, Firefox). Its primary responsibility is to act as a non-intrusive observer that extracts textual content from the user's active browsing session in real-time and renders visual feedback overlays (icons and tooltips) directly onto the web page.
- **The Server (Analysis Backend):** A robust API Gateway built on the FastAPI framework that hosts the system's intelligence. This backend serves as an orchestrator for four

specialized, independent AI modules, allowing for scalable processing and easy integration of future threat detection capabilities.

The software integrates into the user's digital environment by overlaying actionable security insights such as warnings about hate speech, misinformation, privacy risks, or scams directly adjacent to the relevant content. This design ensures that the system serves as an educational tool or "cognitive nudge," empowering users to make informed decisions without removing their autonomy or censoring content.

2.1 Product Functions

The system provides four distinct categories of protection, corresponding to its modular architecture:

1. **Privacy & Operational Security (OpSec) Analysis** The system scans user-drafted text for unintentional self-disclosure. It identifies explicit Personally Identifiable Information (PII) like phone numbers and email addresses, as well as implicit OpSec risks where context (e.g., announcing travel plans) creates a vulnerability.
2. **Misinformation Profiling & Guidance** Rather than simply flagging claims as true or false, the software analyzes text for manipulative rhetorical patterns (e.g., emotional appeals, loaded language). It then delivers educational context tailored to the user's specific media literacy profile, helping them critically evaluate the information.
3. **Toxicity & Hate Speech Recognition** The software evaluates social media posts for toxic language, specifically focusing on nuanced and context-dependent forms of hate speech (e.g., "dog whistles") that often evade standard keyword filters. It distinguishes between constructive criticism and harmful abuse to minimize false alarms.
4. **Scam & Phishing Prevention** The system performs a dual-layer analysis on posts containing links or suspicious offers. It evaluates URLs against known blacklists and heuristics while simultaneously analyzing the accompanying text for psychological triggers of fraud, such as artificial urgency or high-reward promises.

2.1 Product perspective

"The Guardian" occupies a unique niche by addressing **Human-Layer Security** protection against threats that exploit psychological vulnerabilities rather than software vulnerabilities. Below is a comparison with existing solutions currently available in the market:

1. Traditional Antivirus & Anti-Malware Software

- **Existing Functionality:** Tools like Norton, McAfee, or Windows Defender primarily scan for malicious *files* (viruses, trojans) and network intrusions. They focus on infrastructure protection.
- **"The Guardian" Differentiator:** "The Guardian" operates at the **semantic level**, analyzing the *meaning* of text and links. Unlike antivirus software, it detects threats that

contain no malware code but are socially dangerous, such as hate speech, misinformation, or doxing risks.

2. Platform-Native Moderation (e.g., Facebook/X Reporting Tools)
- **Existing Functionality:** Social media platforms employ centralized moderation teams and algorithms to flag or remove content that violates community standards.
- **Limitations:** These systems are **reactive** (acting after content is posted and reported), often opaque, and focus on censorship (removing content) rather than user education.
- **"The Guardian" Differentiator:** This system is **proactive** and **client-side**. It warns the user *before* they engage with harmful content or *before* they post sensitive PII. It empowers the user with educational tooltips ("nudges") rather than censoring the feed, preserving user autonomy.

3. Fact-Checking Websites (e.g., Snopes, PolitiFact)
- **Existing Functionality:** These rely on human researchers to verify claims and publish "True/False" verdicts.
- **Limitations:** They are slow to respond to breaking news, cannot scale to millions of user comments, and provide binary outputs that lack nuance.
- **"The Guardian" Differentiator:** The Misinformation Module uses NLP to detect **manipulative rhetorical patterns** (e.g., "Appeal to Emotion") in real-time. It moves beyond binary fact-checking to provide personalized, profile-aware explanations that improve the user's media literacy.

4. Ad-Blockers & Privacy Extensions
- **Existing Functionality:** Extensions like AdBlock or Ghostery block tracking scripts and advertisements.
- **Limitations:** They prevent third-party data collection but do not stop users from **unintentionally** sharing their own private data (Privacy Paradox).
- **"The Guardian" Differentiator:** The PII & OpSec Module specifically targets **self-disclosure**. It uses NER and Regex to detect when a user is about to post sensitive details (e.g., phone numbers) or contextual risks (e.g., vacation plans implying an empty home) that standard privacy tools completely miss.

5. Scam & Phishing Tools (e.g., Google Safe Browsing)
- **Existing Functionality:** Relies primarily on blacklists of known malicious URLs.
- **"The Guardian" Differentiator:** While existing tools fail against **Zero-Day Phishing** (new links not yet on lists), The Guardian uses **Heuristics** (identifying deep paths and risky extensions) and **Semantic AI** to catch the scam's *intent* in the post text, even if the link is brand new.

2.1.1 System interfaces

Since "The Guardian" is architected as a **Browser Extension (Client)** and a **Web API (Server)**, it utilizes standard high-level interfaces rather than special kernel-level drivers.

Client-Side (Browser Extension)

- **No Direct OS Access:** The client software does not communicate directly with the client operating system (e.g., Windows, macOS, Linux). Instead, it runs within the sandboxed environment of the web browser (Google Chrome, Mozilla Firefox, or Microsoft Edge).
- **Browser Mediation:** All interactions with the underlying hardware or OS resources (such as clipboard access for text analysis or notification system for alerts) are mediated exclusively through the standard **WebExtension APIs** provided by the browser.

Server-Side (Backend API)

- **Standard Process Management:** The backend (FastAPI) interacts with the Server Operating System (recommended Windows) via the **Python Runtime Environment**. It utilizes standard OS interfaces for:
- **File System:** Accessing pre-trained model weights (e.g., Hugging Face cache) and writing temporary logs.
- **Networking:** Binding to TCP ports (standard HTTP/HTTPS ports) to listen for incoming API requests from the extension.
- **Resource Allocation:** Requesting CPU and GPU (CUDA) resources for the execution of Transformer-based NLP models.

2.1.2 User interfaces

The user interface of "The Guardian" follows a **Minimalist and Non-Intrusive** design principle. As a browser extension, it does not rely on a separate dashboard application; instead, it injects subtle visual elements directly into the social media platform's native DOM (Document Object Model).

The UI is built around two primary interaction states: the **Passive Alert** and the **Active Educational Nudge**.

1. Passive Alert Indicator (The "Shield")

- **Principle:** The system must not disrupt the user's browsing flow or block content. Instead of pop-up modals, it uses small, contextual icons.
- **Implementation:**
  - **Consumption Mode (Reading Posts):** When a threat (Hate Speech, Misinformation) is detected in a post, a subtle **color-coded icon** (e.g., a small shield or warning triangle) is overlaid adjacent to the text content.
  - **Creation Mode (Typing Posts):** For the PII & OpSec module, the icon appears next to the text input field in real-time as the user types, indicating a potential self-disclosure risk.
  - **Scam Specific:** Uses a three-tier badge system: SAFE (score 0-29), SUSPICIOUS ((score 30-79), DANGEROUS (score 80-100).

2. Active Educational Tooltip (The "Nudge")

- **Principle:** The system aims to educate rather than censor. Detailed information is only revealed upon user interaction (On-Hover or Click) to avoid visual clutter ("Security Fatigue").
- **Implementation:**
  - **Hover/Click Action:** Interacting with the alert icon opens a localized **tooltip** or **mini-chat window**.
  - **Detailed Explanations:** The tooltip provides a breakdown, e.g., "Suspicious URL structure (Score 90) | AI detected high scam confidence (85%)."
  - **Content:** This panel displays the specific threat type (e.g., "Potential Phishing," "Implicit Hate Speech") and a concise explanation of *why* it was flagged (e.g., "Sharing future travel plans can signal that your home is unattended").

3. Module-Specific Variations

- **Hate Speech & Scam:** Uses standard tooltips to highlight specific "trigger words" or "toxic spans" within the text.
- **Misinformation:** Utilizes a **Conversational/Chat Interface** to explain complex manipulative rhetorical patterns, adapting the complexity of the text to the user's profile.
- **Safe Mode (Scam Module):** Clicking on a "Dangerous" badge triggers a special operation to open the URL in an **Incognito Window**, isolating the session from the user's primary browser data.

2.1.3 Hardware interfaces

Special Hardware Requirements There are **no special hardware devices** or proprietary peripherals required for the operation of "The Guardian". The software is designed to operate entirely on standard consumer-grade computing equipment.

Client-Side Requirements (End User)

- **Device:** A standard personal computer (Laptop or Desktop) capable of running a modern web browser (Google Chrome, Firefox, or Edge).
- **Peripherals:** Standard input/output devices (Keyboard, Mouse, Monitor) are sufficient for interaction.

Server-Side Requirements (Backend Operation)

- **Processing:** A standard cloud-based server instance or local server. While not strictly "special" hardware, **GPU acceleration** (e.g., NVIDIA CUDA-compatible GPUs) is recommended for the backend to ensure low-latency inference (< 500ms) for the Transformer-based NLP models.
- **Memory:** A minimum of 16 GB RAM is recommended to host the four concurrent AI models (Hate Speech, Misinformation, PII/OpSec, Scam) efficiently.

2.1.4 Software interfaces

- The client-side browser extension acts as a software add-on that interfaces directly with the browser's extension engine.
- **Supported Products:** The system interfaces with Chromium-based browsers (e.g., **Google Chrome**, **Microsoft Edge**) and **Mozilla Firefox**.
- **Interface Mechanism:** It utilizes standard **WebExtension APIs** to inject content scripts, access the DOM (Document Object Model) of active tabs, and render UI overlays (icons/tooltips).

Target Products: The system is specifically designed to interface with **X (formerly Twitter)**, **Facebook**, and **Instagram**

Scam Detection: The Scam & Phishing module interfaces with external safety databases to verify suspicious URLs. Specifically, it cross-references links against **PhishTank** and **Google Safe Browsing** blacklists to identify known malicious sites.

- The backend system relies on specific software libraries to execute its AI models.
- FastAPI**:** Acts as the interface framework for the backend API, handling HTTP requests and JSON responses.
- Hugging **Face Transformers:** The primary interface for loading and running the fine-tuned models.
- **spaCy & NLTK:** Used for preprocessing tasks such as tokenization and entity recognition.
- **PyTorch / TensorFlow:** The underlying deep learning frameworks required to execute the model inference.

2.1.5 Communication interfaces

The system relies on network-based communication to facilitate the interaction between the client-side browser extension and the server-side analysis modules.
Primary Protocol: The communication between the Browser Extension (Frontend) and the API Gateway (Backend) is conducted exclusively over **HTTPS (Hypertext Transfer Protocol Secure)**.

- Format**:** All API requests and responses are structured using **JSON (JavaScript Object Notation)**.
- Request **Payload:** The extension sends a JSON object containing the scraped text, relevant metadata (e.g., source platform), and an anonymous user identifier.
- Response **Payload:** The backend returns a structured JSON object containing detection labels (e.g., "Hate Speech," "Misinformation"), confidence scores, and explanatory text for tooltips.

2.1.6 Memory constraints

1. Internal Memory

- Client-Side (Browser Extension):
  - The browser extension is architected to be **"lightweight"**. It operates within the memory space allocated by the host browser (e.g., Chrome, Firefox) for extensions.
  - It must effectively manage DOM manipulation to prevent memory leaks that could slow down the user's browsing experience.
  - Since heavy processing is offloaded to the server, client-side RAM usage is expected to be minimal (typically < 100MB active usage).

- Server-Side (Backend API):
  - The backend performs computationally intensive Natural Language Processing (NLP) tasks. It requires sufficient Random Access Memory (RAM) to load and execute four concurrent Transformer-based models (fine-tuned BERT and DistilBERT).
  - Development specifications recommend a minimum of **16 GB of RAM** to handle the model weights and inference pipeline efficiently.
  - For optimal performance, the backend utilizes **GPU Memory (VRAM)** if available, to accelerate tensor operations using libraries like PyTorch or TensorFlow.

2. External Memory (Secondary Storage)

- **Volatile/Stateless Operation (User Data):**
  - A critical privacy constraint is the **absence of persistent storage for user content**. The system is designed to be **stateless**: text scraped from social media is processed in memory and discarded immediately after analysis.
  - The system explicitly forbids the logging or retention of user-submitted text databases.
- **Persistent Storage (System Data):**
  - **Client:** Minimal storage is required on the user's device, limited to the extension's code files (JavaScript/HTML/CSS) and small configuration files for user preferences (e.g., alert sensitivity settings)

2.1.7 Operations

The system is designed to minimize the operational burden on the user, operating primarily in a passive, background mode. However, specific normal and special operations are required to interact with the security features effectively.

1. Normal Operations (Routine Interaction) These are the standard tasks a user performs during regular social media usage.

- **Passive Monitoring:** The user browses social media feeds (e.g., scrolling through Facebook or X) without taking specific action. The system automatically scrapes and analyzes visible content in real-time.
- **Visual Alert Recognition:** The user observes color-coded icons (e.g., Red for high risk, Orange for caution) overlaid next to posts or comments, indicating detected threats like hate speech or misinformation.
- **Tooltip Interaction:** The user hovers over or clicks on a warning icon to reveal a tooltip. This operation provides immediate, concise context about *why* the content was flagged (e.g., "Loaded Language Detected").
- **Secure Text Entry:** When drafting posts or comments, the user types normally. The system monitors input in real-time and displays an immediate warning if PII (e.g., phone numbers) or OpSec risks (e.g., travel plans) are detected before the user hits "Post".

2. Special Operations (Advanced & On-Demand Interaction) These operations are performed less frequently or in response to high-risk scenarios.

- **Manual Text Verification:** The user manually highlights a specific segment of text on a page to trigger an on-demand analysis, specifically for verifying potential misinformation that might have been missed by the auto-scan.
- **Chatbot Engagement:** Upon encountering a complex misinformation alert, the user initiates a chat session via the extension to receive a detailed, personalized debunking or explanation adapted to their profile.
- **Safe Mode Navigation:** When a suspicious URL is flagged by the Scam & Phishing module, the user may choose to open the link in **"Safe Mode"** (an isolated or read-only preview state) to inspect the content without executing malicious scripts.

2.1.8 Site adaptation requirements

To execute the browser extension, the user's device must meet the following prerequisites:

- **Operating System:** Platform-independent; compatible with Windows (10/11), macOS (10.15+), or Linux distributions, provided a supported browser is installed.
- **Web Browser:** A modern, Chromium-based browser (e.g., **Google Chrome**, **Microsoft Edge**, **Brave**) or **Mozilla Firefox** (v90+) supporting the WebExtensions API.
- **Network Connectivity:** A stable broadband internet connection is required. The extension must communicate with the backend API over **HTTPS** to perform real-time text analysis.
- **Permissions:** The user must grant the extension permission to "Read and change all your data on the websites you visit" (host permission) for the targeted social media domains (Facebook, X/Twitter, Instagram) to enable DOM scraping.

• Hardware **Resources:**

- **RAM:** Minimum **16 GB** is recommended to load all four models (Hate Speech, Misinformation, PII/OpSec, Scam) into memory concurrently.

- **Compute:** A multi-core CPU is sufficient for development, but a **CUDA-enabled GPU** is recommended for production environments to maintain the <500ms response time target.

- External **Data Access:**

  - The server requires outbound internet access to query external threat intelligence APIs (e.g., **PhishTank**, **Google Safe Browsing**) for the Scam & Phishing module.

2.2 Product functions

"The Guardian" provides a suite of proactive cybersecurity functions designed to mitigate human-layer threats on social media. These functions are categorized into four primary modules, operating concurrently to analyze user-generated content and external interactions.

1. Privacy & Operational Security (OpSec) Risk Detection This function monitors the user's input fields and drafted text in real-time to prevent unintentional data leakage.

- **Structured PII Detection:** Utilizes Regex-based pattern matching to instantly identify and flag explicit personally identifiable information (PII) such as phone numbers, email addresses, and National Identity Card (NIC) numbers before they are posted.
- **Unstructured Entity Recognition:** Deploys a hybrid Named Entity Recognition (NER) model to detect context-sensitive entities like home addresses or specific location names.
- **Contextual OpSec Analysis:** Applies a rule-based engine to identify "contextual" vulnerabilities where the combination of entities creates a risk (e.g., combining "Location" + "Future Date" + "Travel Intent" signals that the user's home may be unattended).

2. Misinformation Profiling & Guidance This function moves beyond binary fact-checking to educate users about manipulative rhetoric, adapted to their specific level of media literacy.

- **Manipulative Pattern Recognition:** Analyzes text using fine-tuned Transformers to detect linguistic indicators of misinformation, such as "Appeal to Emotion," "Loaded Language," or "Conspiracy Framing," rather than solely verifying claims against a database.
- **User Profiling Mechanism:** Captures user metadata (e.g., education level, prior awareness) to build a dynamic user profile. This profile determines the complexity and style of the educational feedback provided.
- **Adaptive Chat Interface:** Upon detection, the system engages the user via a conversational chatbot that provides a personalized explanation or "nudge" tailored to the user's profile, promoting critical thinking rather than censorship.

3. Hate Speech & Toxicity Detection This function evaluates social media posts to identify harmful language, distinguishing between legitimate criticism and abuse.

- **Implicit & Explicit Detection:** Uses fine-tuned BERT/DistilBERT models to classify text into categories (Neutral, Offensive, Hate Speech). It is specifically trained to recognize subtle, context-dependent toxicity and "dog whistles" that evade standard keyword filters.
- **Explainable Alerting:** Provides interpretable feedback by highlighting the specific "toxic spans" or trigger words within the text that contributed to the classification, helping users understand why a post was flagged.

4. Scam & Phishing Detection - The system secures the user against financial fraud and social engineering through a **Hybrid Dual-Layer Analysis**:

- **Dual-Path Analysis:** simultaneously executes:
    - **Path A (URL Analysis):** Cross-references extracted URLs against external threat intelligence blacklists (e.g., PhishTank, Google Safe Browsing) and heuristic rules (e.g., homoglyph attacks).
- **Path B (Text Analysis):** Analyzes the accompanying post text for psychological triggers of fraud, such as artificial urgency, authority, or high-reward promises.

  Function 4.1: Deterministic URL Heuristics:

- **Subdomain Analysis:** Penalizes URLs with 3+ dots (+20 risk).
- **Path Depth Analysis:** Penalizes deep directory structures (e.g., `/folder/sub/page.html`) often used to hide phishing kits (+40 risk).
- **Binary/Risky Extension Check:** Flags `.exe`, `.php`, and `.zip` files (+50 to +95 risk).

Function 4.2: Context-Aware AI Inference:

- Analyzes the surrounding post text using a fine-tuned DistilBERT model.
- Utilizes **URL Tokenization** to ensure the model identifies "scam language" (e.g., "claim reward," "account suspended") regardless of the specific link used.
- **Function 4.3: Max Risk Fusion Logic:**
- The backend calculates: `Final_Score = max(URL_Heuristic_Score, AI_Text_Score)`.
- This prevents a "Dangerous" URL from being downgraded by "Safe" appearing text.

Function 4.4: Super Whitelist Short-Circuit:

- Identifies major platforms (Google, X, Facebook) to bypass AI analysis, reducing CPU load and preventing false positives on trusted domains.

2.3 User characteristics

The user base for "The Guardian" is broad, encompassing the general population of social media users. However, for the purpose of requirement specification, the users are categorized into three distinct personas based on their technical expertise and vulnerability levels.

1. The "Everyday" Social Media User (Novice to Intermediate)

- **Profile:** This is the primary demographic. They use platforms like Facebook, X (Twitter), and Instagram daily for entertainment and social connection.
- **Technical Skill:** Low to Medium. They are comfortable browsing the web but lack deep technical knowledge of cybersecurity concepts (e.g., they may not recognize a homograph phishing attack or understand how metadata reveals location).
- **Behavioral Traits:** They tend to scroll quickly, often engaging with content emotionally rather than critically. They are susceptible to "click-bait," artificial urgency in scams, and manipulative rhetoric in misinformation.
- **Requirement:** They require a **"Zero-Configuration"** experience where protection is passive and automatic. Alerts must be simple (e.g., traffic-light colors) and non-technical to avoid confusion.

2. The Vulnerable User (At-Risk Demographics)

- **Profile:** This group includes individuals specifically targeted by human-layer threats, such as the elderly (susceptible to financial scams), younger users (susceptible to radicalization or cyberbullying), and marginalized communities (targets of hate speech).
- **Technical Skill:** Novice. They may have limited media literacy and struggle to distinguish between credible news and fabricated content.
- **Requirement:** They benefit most from the **Educational/Chatbot** component, which patiently explains *why* content is dangerous, helping to build their long-term resilience and media literacy.

3. The Privacy-Conscious User (Intermediate)

- **Profile:** Users who are concerned about their digital footprint but lack the specialized Operational Security (OpSec) knowledge to protect themselves effectively.
- **Technical Skill:** Intermediate. They know basic privacy settings but often commit "unintentional self-disclosure" (e.g., posting a phone number or announcing vacation plans publicly).
- **Requirement:** They need proactive **"Nudges"** during the content creation process (typing) to prevent accidental data leakage before it happens.

2.4 Constraints

These constraints impose limits on the design choices and implementation strategies available to the development team.

1. Regulatory & Privacy Constraints
    - **Zero-Data Retention:** The system must strictly adhere to a **stateless** architecture for user content. To protect user privacy and avoid liability for handling sensitive data, no user-submitted text (posts, comments, or messages) may be stored, logged, or retained on the backend servers after the analysis is complete.
    - **Anonymity:** All data transmitted to the backend must be stripped of personal identifiers. The system must use anonymous user identifiers rather than linking analysis to specific social media accounts.
2. Technical & Hardware Constraints
    - **Browser Sandboxing:** The client-side extension must operate within the strict security sandbox of the WebExtensions API (Manifest V3). It cannot access the user's file system or OS kernel directly; all interactions must be mediated by the browser.
    - **Latency Thresholds:** To ensure the user experience remains smooth, the total round-trip time for analysis (Scrape $\rightarrow$ API $\rightarrow$ Alert) is constrained to **under 500 milliseconds** for standard text inputs (<280 characters).
    - **Consumer-Grade Client Hardware:** The frontend must run efficiently on standard consumer laptops with limited resources. Memory usage for the extension is constrained to avoid slowing down the host browser.
3. Economic & Resource Constraints
    - **Zero-Budget Implementation:** The project is constrained to a budget of zero financial cost. The implementation must exclusively utilize **Open-Source Software** (Python, FastAPI, Hugging Face) and **Free-Tier** cloud resources or local university hardware.
    - **Computational Limits:** Due to the lack of enterprise-grade cloud GPUs, model fine-tuning and inference must be optimized for efficiency (e.g., using **DistilBERT** instead of larger models like GPT-4) to run on available personal or lab hardware.

2.5 Assumptions and dependencies

The successful implementation and operation of the software rely on the following assumptions and external factors.

1. Dependencies on External Systems

- **Social Media DOM Structure:** The scraping functionality depends on the stability of the HTML/DOM structures of target platforms (Facebook, X, Instagram). Significant updates to these platforms' frontend code may require updates to the extension’s content scripts.
- **Threat Intelligence APIs:** The Scam & Phishing module depends on the availability and uptime of third-party APIs, specifically **PhishTank** and **Google Safe Browsing**, to verify malicious URLs.
- **Pre-trained Models:** The system relies on the availability of pre-trained Transformer models (BERT, DistilBERT) and tokenizers hosted by **Hugging Face**.

2. Operational Assumptions

- **Internet Connectivity:** It is assumed that the end-user has a stable, continuous internet connection. As a cloud-based analysis system, "The Guardian" cannot function in an offline environment.
- **HTTPS Encryption:** It is assumed that the deployment environment supports valid SSL/TLS certificates to ensure secure HTTPS communication between the extension and the backend.
- **Browser Compatibility:** It is assumed that users are running updated versions of Chrome, Edge, or Firefox that support the latest WebExtensions API standards.

3. Data Assumptions

- **Dataset Representativeness:** It is assumed that the training datasets (LIAR, CoAID, Davidson, HateXplain) are sufficiently representative of real-world social media language to allow the models to generalize effectively.

2.6 Apportioning of requirements

Based on the project's Work Breakdown Structure (WBS), the requirements will be implemented in the following phased order.

Phase 1: Core Intelligence (Parallel Development)

- *Priority: Critical*
- Implementation of the four independent detection algorithms.
    1. **PII & OpSec Module:** Implementing the hybrid NER and Regex rule engine.
    2. **Misinformation Module:** Training models on LIAR/CoAID and developing the User Profiling logic.
    3. **Hate Speech Module:** Fine-tuning BERT & HateXplain for toxic span detection.
    4. **Scam Module:** Developing the URL scanner and text-urgency classifiers.

Phase 2: Backend Orchestration

- *Priority: High*
- Development of the **FastAPI Gateway**.
- Integration of the four core modules into a unified API endpoint structure.
- Implementation of JSON response formatting and error handling.

Phase 3: Frontend & User Interface

- *Priority: High*
- Development of the **Browser Extension** shell (Manifest, Background Scripts).
- Implementation of the **DOM Scraper** to extract text from social media.
- Integration of visual assets: Warning Icons, Tooltips, and the Chatbot Interface.

Phase 4: Advanced Features & Refinement

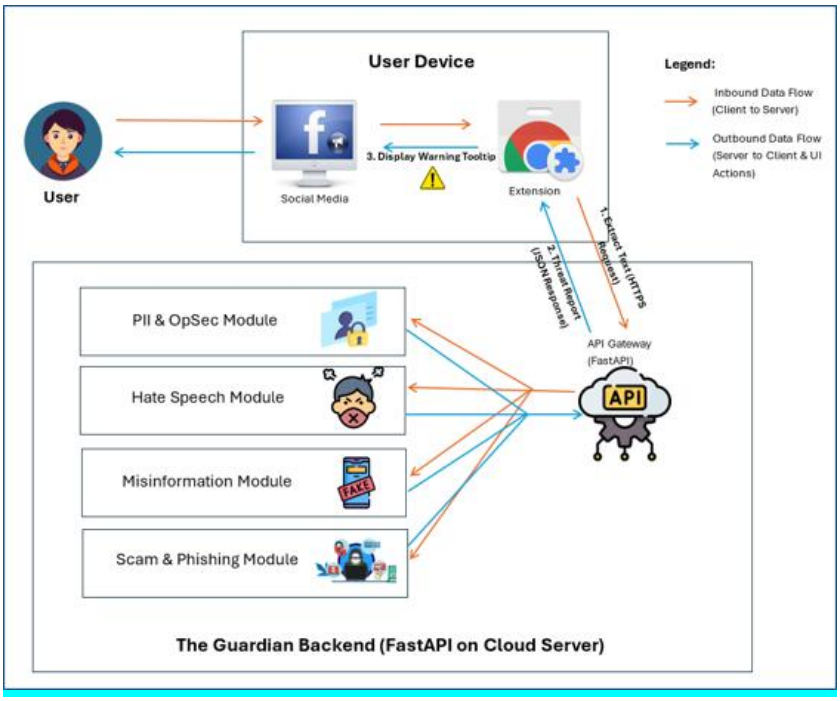
- *Priority: Medium*
- Implementation of **"Safe Mode"** containerization for viewing suspicious links.
- Refinement of the **Explanation Generator** (Chatbot) based on initial user testing.
- Optimization of model latency to meet the <500ms NFR.

3. Specific requirements[1] (for Software Dev. Oriented Projects - SRS)

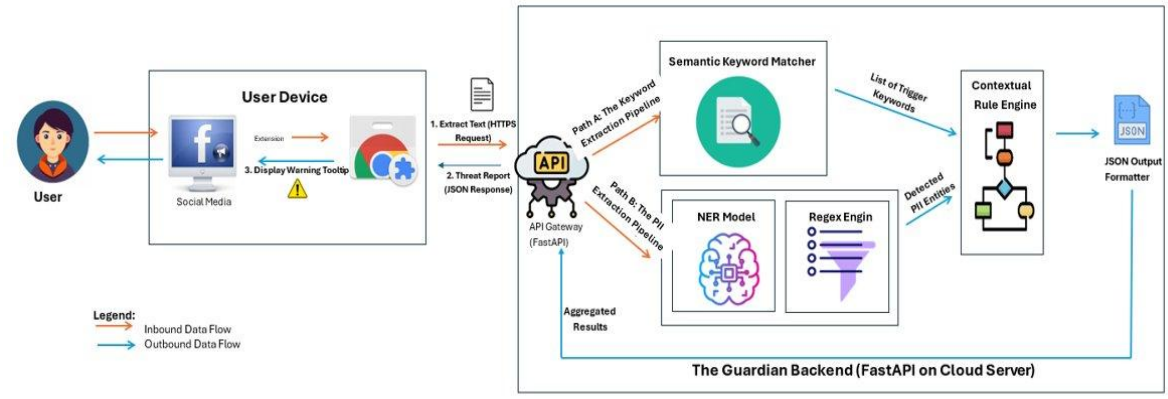
3.1 External interface requirements

3.1.1 User interfaces

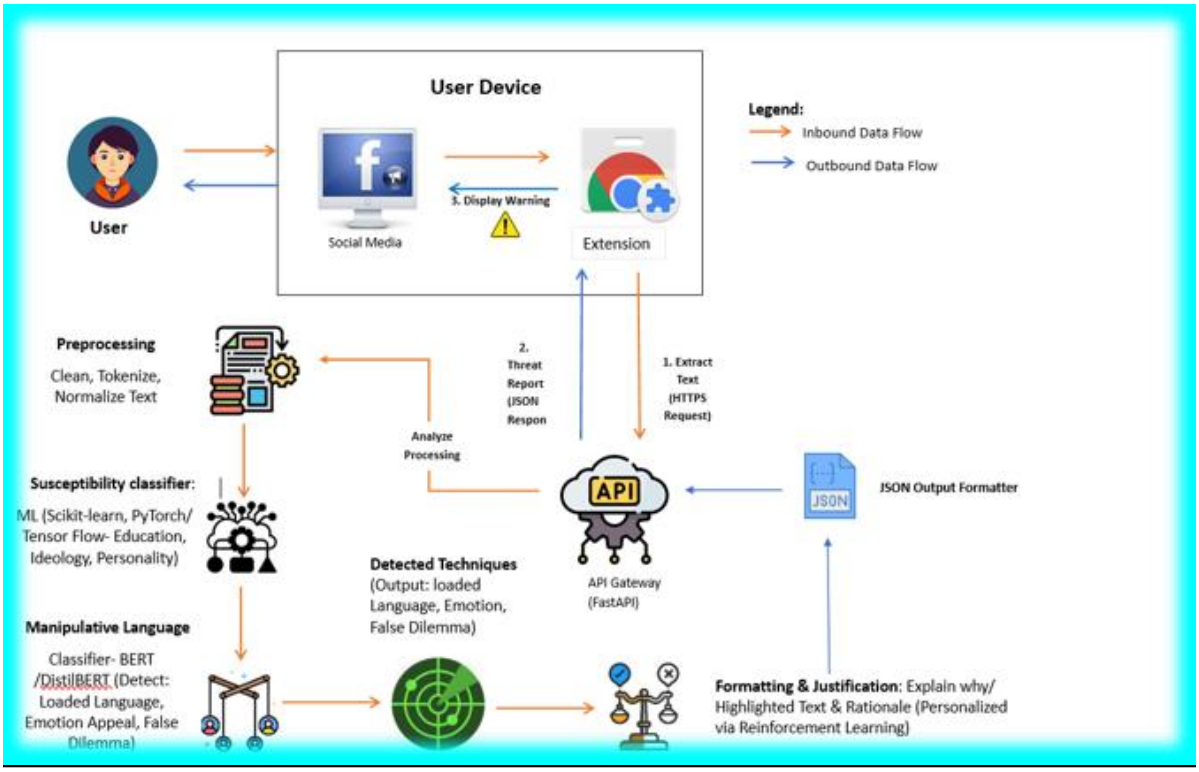
The Guardian: Overall System Diagram



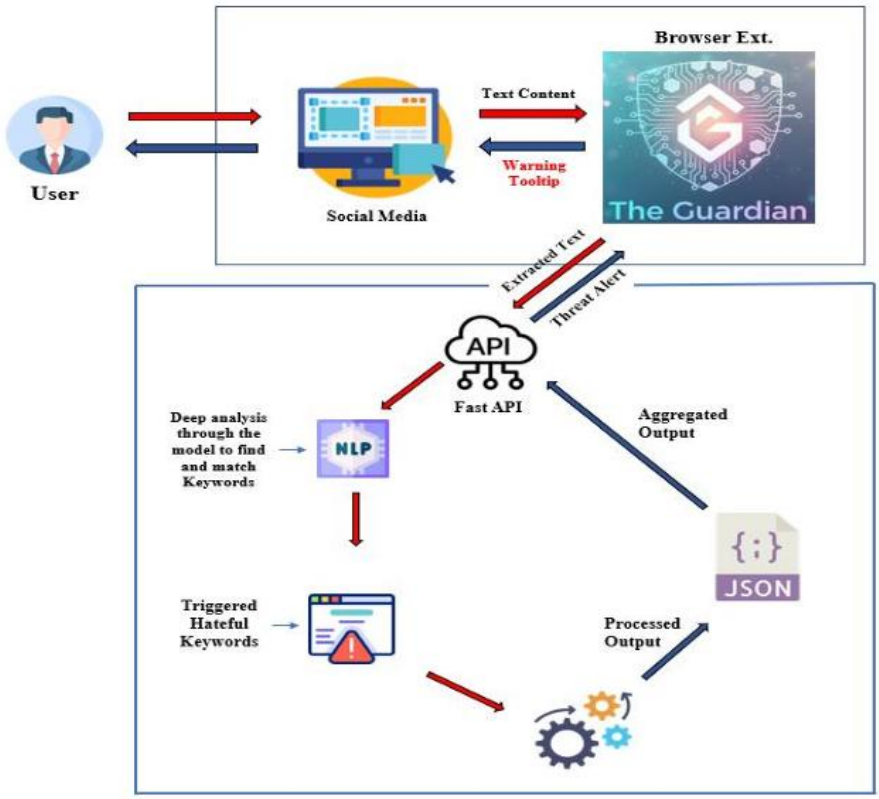
1. PII & OpSec Risks: System Design



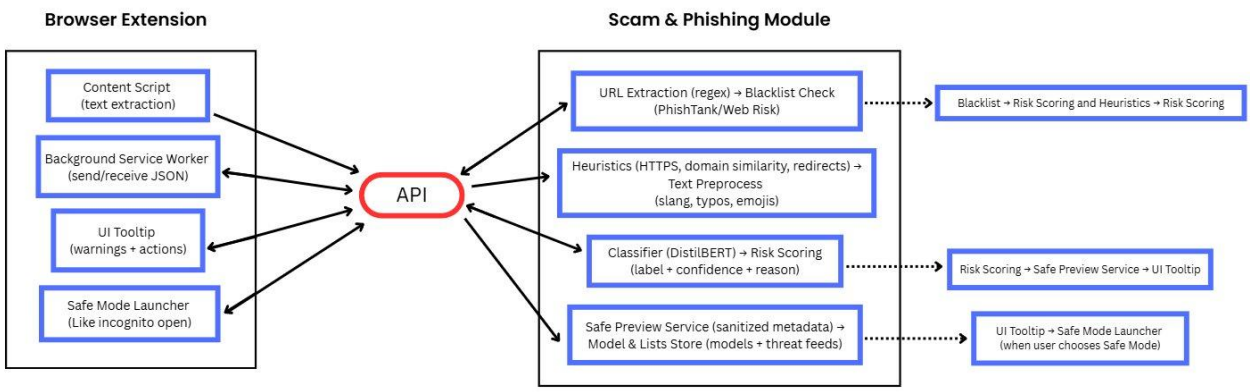
2. Misinformation: System Design



3. Hate Speech: System Flow



4. Phishing & Scam Module: System Design



3.1.3 Software interfaces

"The Guardian" functions as a bridge between the end-user's browser and the social media platforms they visit. It relies on specific interfaces with external software components to execute its detection logic.

1. Web Browsers (Host Environment)

- The client-side software is a browser extension that interfaces directly with the host browser's extension engine via the **WebExtensions API**.
- **Supported Products:** The system must interface with **Google Chrome**, **Microsoft Edge**, and **Mozilla Firefox**.
- **Interaction:** The extension uses browser APIs to inject content scripts into the active tab, access the DOM (Document Object Model) for text scraping, and render the UI overlays (icons and tooltips).

2. Social Media Platforms (Target Systems)

- The software interfaces with the frontend structure of specific social media web applications.
- **Target Products:** The system is designed to interface with **X (formerly Twitter)**, **Facebook**, and **Instagram**.
- **Interaction:** It reads the HTML structure of these sites to identify and extract user-generated content (posts, comments) in real-time.

3. External Threat Intelligence APIs

- The **Scam & Phishing Module** interfaces with external third-party security databases to validate suspicious URLs found in posts.

- **External Services:** The system queries **PhishTank** and **Google Safe Browsing** APIs to check if a specific URL appears on known blocklists.
- **Misinformation Support:** The system may cross-reference content with datasets like **LIAR** and **CoAID** to support classification accuracy.

4. Backend Libraries & Frameworks

- The server-side application interfaces with various Python-based software libraries to manage the AI pipeline.
- **FastAPI:** The web framework used to handle HTTP requests and route them to the appropriate module.
- **Hugging Face Transformers:** The library used to load and interface with the pre-trained BERT and DistilBERT models.
- **PyTorch / TensorFlow:** The underlying deep learning frameworks that execute the model inference.
- **spaCy:** Used for Natural Language Processing tasks like tokenization and entity recognition.

3.1.4 Communication interfaces

The system is a distributed client-server application that relies on standard network communication protocols to function.

1. Communication Protocols

- **HTTPS (Hypertext Transfer Protocol Secure):** All communication between the Browser Extension (Client) and the Backend API (Server) is conducted exclusively over HTTPS. This ensures that the text content being analyzed is encrypted in transit, preventing interception by third parties.
- **JSON (JavaScript Object Notation):** The primary data exchange format. The client sends a JSON payload containing the scraped text and user ID, and the server responds with a JSON object containing detection labels and confidence scores.

2. Network & Connectivity

- **Internet Connection:** The system requires a continuous, stable broadband internet connection to function. It cannot operate offline as the heavy AI processing is offloaded to the cloud server.

3.3 Performance requirements

The requirements for "The Guardian" are organized around the following primary classes, which encapsulate the system's domain logic and data structures.

1. Analysis Core (Backend Modules)

These classes represent the primary intelligence of the system, residing on the server.

- **Class:** `PII_OpSecScanner`
  - **Description:** Scans drafted text for structured PII (via Regex) and unstructured entities (via NER) to prevent self-disclosure. It also evaluates contextual logic (Entity + Time = Risk).
  - **Importance: Essential**
  - **Associated Requirements:** Phone/Email detection, travel risk logic, real-time input monitoring.
- **Class:** `MisinformationAnalyzer`
  - **Description:** Evaluates content for manipulative rhetorical patterns (e.g., emotional appeals, logical fallacies) and cross-references claims with known misinformation datasets (LIAR, CoAID).
  - **Importance: Essential**
  - **Associated Requirements:** Pattern recognition, confidence scoring, dataset lookup.
- **Class:** `HateSpeechDetector`
  - **Description:** Responsible for analyzing text to identify explicit hate speech, offensive language, and implicit "dog whistles" using fine-tuned Transformer models.
  - **Importance: Essential**
  - **Associated Requirements:** Detection of toxic spans, classification of severity, differentiation between criticism and abuse.
- **Class:** `ScamShield`
  - **Description:** Manages the dual-path analysis of suspicious posts, coordinating the URL verification (against PhishTank/Google Safe Browsing) and text urgency analysis.
  - **Importance: Essential**
  - **Associated Requirements:** URL extraction, blacklist querying, urgency detection.

2. User Interaction & Visualization (Frontend)

These classes manage how the user interfaces with the system within the browser.

- **Class:** `ContentScraper`
  - **Description:** A background worker responsible for detecting DOM changes, extracting visible text from social media feeds, and packaging it for analysis.
  - **Importance: Essential**
  - **Associated Requirements:** DOM observation, text extraction, scroll event handling.

- **Class:** `AlertOverlay`
    - **Description:** The UI component responsible for injecting visual indicators (icons, highlights) into the web page adjacent to the flagged content without breaking the page layout.
    - **Importance: Essential**
    - **Associated Requirements:** Icon rendering, non-intrusive positioning, color-coding (Red/Orange/Green).
- **Class:** `EducationalChatbot`
    - **Description:** An interactive interface that triggers when a user engages with a Misinformation alert, providing a conversational explanation tailored to the user's profile.
    - **Importance: Desirable**
    - **Associated Requirements:** Chat interface rendering, dialogue management, adaptive explanation text.

- **Class:** `SafeModeContainer`
    - **Description:** An isolated viewing environment (e.g., a sandboxed iframe or read-only preview modal) that allows users to inspect suspicious links flagged by the Scam module without risk of execution.
    - **Importance: Desirable**
    - **Associated Requirements:** Link interception, sandboxed rendering, read-only preview generation.

3. Data & Context Objects

These classes represent the data structures exchanged between the client and server.

- **Class:** `ThreatReport`
    - **Description:** The standardized JSON object returned by the API, containing the detection label, confidence score, flagged text segments, and explanation strings.
    - **Importance: Essential**
    - **Associated Requirements:** JSON serialization, error handling, standardized API response format.
- **Class:** `UserProfile`
    - **Description:** A metadata object storing the user's media literacy level, education background, and domain knowledge. This is used specifically by the Misinformation module to tailor explanations.
    - **Importance: Desirable**
    - **Associated Requirements:** Metadata capture, profile storage (local), query parameter injection.
- **Class:** `UserPreferences`
    - **Description:** Manages user-defined settings, such as alert sensitivity levels (Low/Medium/High) and module toggles (e.g., "Disable PII Detection").
    - **Importance: Optional**

- **Associated Requirements:** Settings menu UI, local configuration storage, sensitivity logic application.

3.3 Performance requirements

The system must meet specific performance benchmarks to ensure the user experience remains seamless and non-intrusive, given that it operates in real-time within a web browser.

1. Speed & Latency

- **Real-Time Analysis Latency:** The total round-trip time (RTT) for processing a standard social media post (under 280 characters) shall not exceed **500 milliseconds** under normal load conditions.
- **Maximum Response Time:** For longer text segments (>500 words) or complex multi-module analysis (e.g., concurrent Misinformation and PII checks), the system shall return a response within **2 seconds**.
- **UI Rendering Speed:** The browser extension shall inject the visual alert icon into the DOM within **100 milliseconds** of receiving the API response to prevent visual "pop-in" or layout shifts.

2. Throughput & Scalability

- **Concurrency:** The backend API shall support handling **100 concurrent analysis requests** per active session without failure.
- **Load Scaling:** The system shall support a user base of up to **1,000 concurrent users** with less than a **5% degradation** in average response time.
- **Rate Limiting:** To prevent abuse and ensure fair resource allocation, the API shall enforce a rate limit (e.g., 60 requests per minute per anonymous user ID).

3. Memory & Resource Usage

- **Server-Side RAM (Static):** The backend infrastructure requires a minimum allocation of **16 GB of RAM** to keep the four Transformer-based models (Hate Speech, Misinformation, PII, Scam) loaded in active memory for instant inference.
- **Server-Side VRAM (Dynamic):** If GPU acceleration is enabled, the system requires approximately **4-8 GB of VRAM** for efficient batch processing of text tensors.
- **Client-Side Footprint:** The browser extension is constrained to use less than **100 MB of client RAM** during active scanning to ensure it does not slow down the host browser or other tabs.

4. Reliability & Availability

- **Service Availability:** The backend API shall target an uptime of **99.9%** during operational hours.

- **Fallback Mode:** In the event of a server timeout or network failure, the extension shall fail gracefully by suppressing alerts rather than blocking the user's content or displaying error messages.

3.4 Design constraints

The design of "The Guardian" is limited by the following constraints, which impact architectural decisions and technology selection.

1. Standard Compliance & Privacy (Ethical Constraints)

- **Stateless Architecture:** To comply with ethical privacy standards and minimize liability, the backend must be designed as a **stateless system**. User-submitted text must be processed in volatile memory and immediately discarded after the API response is sent. The system is strictly prohibited from building a database of user messages.
- **Anonymity Preserving:** The system must use randomized, anonymous session identifiers. It cannot access or store the user's actual social media credentials (username/password) or link analysis results to a specific real-world identity.
- **Bias Mitigation:** The Hate Speech module is constrained by the requirement to actively mitigate algorithmic bias. The model training pipeline must include fairness evaluation metrics (e.g., demographic parity) to ensure it does not unfairly flag non-toxic speech from marginalized groups (e.g., AAVE) as hate speech.

2. Hardware & Resource Constraints

- **Zero-Budget Requirement:** The project must be implemented using exclusively **Open Source** software and free-tier resources. This prohibits the use of paid enterprise APIs (like GPT-4 or Google Cloud paid tiers) for the core functionality.
- **Computational Limitations:** The backend must run on standard university lab hardware or consumer-grade servers. Consequently, the NLP models are constrained to **"Distilled" or "Lightweight" architectures** (e.g., DistilBERT instead of BERT-Large) to ensure they can run efficiently without requiring high-end enterprise GPUs.

3. Software & Technology Constraints

- **Browser Sandbox (Manifest V3):** The frontend design is strictly limited by the security model of modern web browsers (Chrome Manifest V3). The extension cannot execute arbitrary code outside the browser context, access the file system freely, or use blocking web request methods that degrade browser performance.
- **DOM-Dependency:** The scraping mechanism is constrained by the frontend HTML structure of the target platforms (Facebook, X, Instagram). The design must be modular to allow for rapid updates, as the system will break if these platforms change their CSS class names or DOM hierarchy.

4. Operational Constraints

- **Connectivity Dependency:** The system is constrained to **Online-Only Operation**. Due to the size of the Transformer models, inference cannot be performed locally on the client's browser; therefore, the system will cease to function if the user loses internet connectivity or if the backend server is unreachable.

3.5 Software system attributes

3.5.1 Reliability

The system is designed to maintain operational stability even under adverse conditions or during partial component failures.

- **Fail-Safe Operation:** In the event of a backend API failure or network timeout, the client-side extension shall **fail gracefully**. It must silently abort the analysis request without crashing the host browser or blocking the user's access to the social media content.
- **Error Handling:** The FastAPI backend implements robust error handling to manage malformed JSON requests or unexpected model behaviors, ensuring that a single faulty request does not bring down the entire service.
- **False Positive Mitigation:** The system prioritizes reliability in detection accuracy. The Hate Speech module includes specific tuning to reduce false positives on legitimate critiques to below 5%, ensuring users do not lose trust in the system's alerts.

The system is designed to maintain operational stability even under adverse conditions or during partial component failures.

- **Fail-Safe Operation:** In the event of a backend API failure or network timeout, the client-side extension shall **fail gracefully**. It must silently abort the analysis request without crashing the host browser or blocking the user's access to the social media content.
- **Error Handling:** The FastAPI backend implements robust error handling to manage malformed JSON requests or unexpected model behaviors, ensuring that a single faulty request does not bring down the entire service.
- **False Positive Mitigation:** The system prioritizes reliability in detection accuracy. The Hate Speech module includes specific tuning to reduce false positives on legitimate critiques to below 5%, ensuring users do not lose trust in the system's alerts.

3.5.2 Availability

The system relies on high availability to provide real-time protection.

- **Uptime Target:** The backend API Gateway is designed for continuous operation, targeting an uptime of **99.9%** during active usage hours. This is facilitated by cloud-based hosting (e.g., AWS or Google Cloud) which provides redundancy.
- **Scalable Architecture:** The modular design allows individual detection modules (e.g., the Scam module) to be scaled independently. If the Hate Speech module experiences a traffic spike, it should not degrade the availability of the PII detection service.

3.5.3 Security

As a cybersecurity product, the system adheres to strict security-by-design principles.

- **Data Transmission:** All communication between the browser extension and the backend API is encrypted using **TLS/SSL (HTTPS)** to prevent Man-in-the-Middle (MitM) attacks during the transmission of sensitive user text.
- **Stateless Processing:** The backend strictly enforces a **"No-Log" policy** for user content. Analyzed text is held in volatile memory only for the duration of the inference process and is immediately discarded thereafter. No databases of user posts are maintained.
- **Anonymity:** The system does not require user authentication (login) to function. Analysis requests are tagged with randomized session IDs rather than persistent user accounts, ensuring that behavioral profiles cannot be linked to real-world identities.

3.5.4 Maintainability

The system architecture prioritizes ease of updates and modular maintenance.

- **Modular Coupling:** The four AI modules (Hate Speech, Misinformation, PII, Scam) are loosely coupled. This allows the development team to update, retrain, or replace the underlying model of one module (e.g., swapping DistilBERT for RoBERTa in the Hate Speech module) without requiring a redeployment of the entire system or updates to the client-side extension.
- **Standardized Interfaces:** The use of a unified JSON schema for API responses ensures that the frontend extension does not need code changes when backend logic is improved, provided the output format remains consistent.

3.6 Other requirements

3.6.1 Ethical & Fairness Requirements

- **Bias Mitigation:** The Hate Speech Detection module includes a strict requirement for **Algorithmic Fairness**. The model must be evaluated against demographic parity metrics to ensure it does not exhibit bias against specific dialects (e.g., AAVE) or marginalized

groups. Mitigation techniques such as re-weighting training data must be applied if bias is detected.

- **User Autonomy:** The system must adhere to the principle of "Nudging" rather than blocking. It is explicitly required *not* to censor or hide content automatically; the final decision to engage with or view content must always remain with the user.

3.6.2 Localization & Language Requirements

- **Multilingual Support:** While the primary focus is English, the system architecture requires support for **multilingual detection** capability. Specifically, the Hate Speech module is designed to eventually handle code-mixed languages (e.g., Hindi-English) and non-English contexts (e.g., Turkish) by incorporating diverse datasets.

3.6.3 Legal Compliance

- **Data Protection Regulations:** The system's design (specifically the PII module and stateless architecture) is intended to be compliant with data protection principles akin to GDPR/PDPA, by ensuring that no personal data is collected or stored without explicit user consent (opt-in for data sharing).